

Sharon - Section 6:

Initial Network:

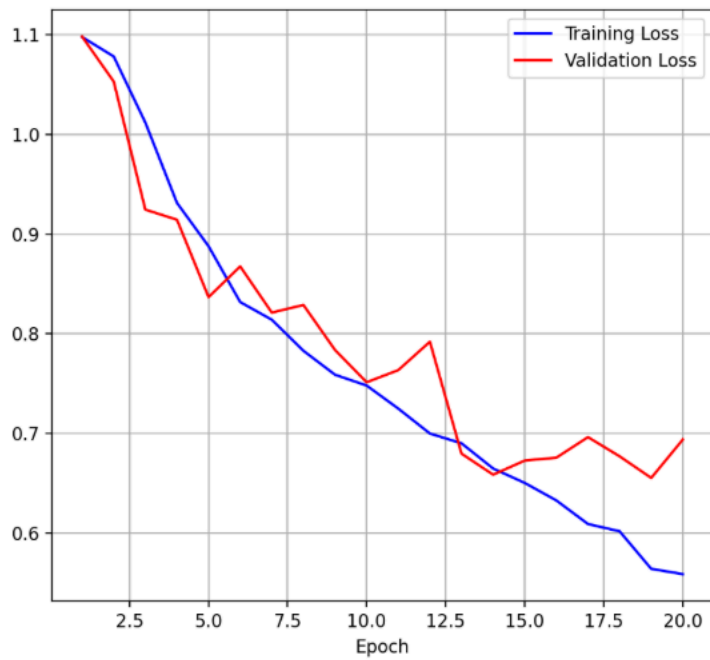
```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 150, 150, 3)	0
conv2d (Conv2D)	(None, 148, 148, 16)	448
max_pooling2d (MaxPooling2D)	(None, 74, 74, 16)	0
conv2d_1 (Conv2D)	(None, 72, 72, 32)	4640
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 32)	0
conv2d_2 (Conv2D)	(None, 34, 34, 64)	18496
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 64)	0
conv2d_3 (Conv2D)	(None, 15, 15, 32)	18464
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 32)	0
flatten (Flatten)	(None, 1568)	0
dense (Dense)	(None, 64)	100416
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 3)	195

```
=====  
Total params: 142,659  
Trainable params: 142,659  
Non-trainable params: 0
```

Total Params: 142,659 (<150k)

Plot: Loss

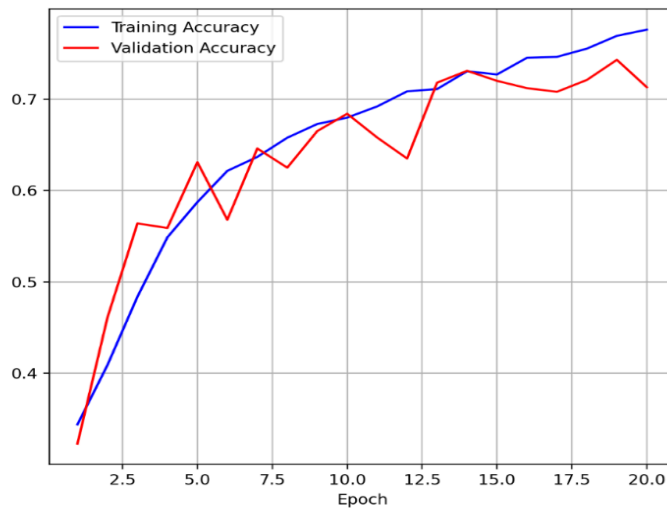


Result:

The Overfitting point is at Epoch 19 (~0.65)

```
Epoch 16/20
32/32 [=====] - 11s 341ms/step - loss: 0.6330 - accuracy: 0.7452 - val_loss: 0.6759 - val_accuracy: 0.7120
Epoch 17/20
32/32 [=====] - 11s 350ms/step - loss: 0.6094 - accuracy: 0.7462 - val_loss: 0.6965 - val_accuracy: 0.7080
Epoch 18/20
32/32 [=====] - 12s 377ms/step - loss: 0.6021 - accuracy: 0.7552 - val_loss: 0.6772 - val_accuracy: 0.7210
Epoch 19/20
32/32 [=====] - 13s 391ms/step - loss: 0.5645 - accuracy: 0.7692 - val_loss: 0.6556 - val_accuracy: 0.7430
Epoch 20/20
32/32 [=====] - 13s 412ms/step - loss: 0.5592 - accuracy: 0.7760 - val_loss: 0.6941 - val_accuracy: 0.7130
```

Plot: Accuracy



Test Set:

Test Accuracy: 72.98%

Best validation accuracy: 74.3% at epoch 19

Overfitting point: epoch 19 (lowest validation loss: 0.6556)

```
* Evaluating basic_model
30/30 [=====] - 4s 129ms/step - loss: 0.6549 - accuracy: 0.7298
```

Report on Hyperparameter Optimization Strategy:

- What I changed → (Dropout) and (RMSprop learning_rate)
- (Dropout) = I increased it from 0.3 to 0.5
- (RMSprop learning_rate) = I basically cut it in half from 0.001 to 0.0005
- Affected Accuracy:
 - When I increased the dropout, it stopped the model from memorizing specific pixels and forced the network to distribute weight mapping across general facial structures. And by decreasing the learning rate, it made the training smoother
 - With both changes, it made the model more stable than the baseline version. Even though there were small mistakes the model was able to quickly fix itself. It successfully stopped overfitting from happening until Epoch 19, reaching a peak accuracy of ~74.5% which is about the 70% target.

Best learned model as a .keras file (it's in the results):

results/basic_model_20_epochs_timestamp_1785630672.keras

Tic-Tac-Toe Section 7:

Traces of moves:

```
| | | |  
| | | |  
| | | |
```

Turn: X

Player X took position (1, 0).

```
| | | |  
| X | | |  
| | | |
```

Turn: O

Emotion detected as neutral (row 0).

Emotion detected as neutral (col 0).

Manual Text Override -> index 0

Player O took position (0, 0).

```
| O | | |  
| X | | |  
| | | |
```

Turn: X

Player X took position (2, 2).

```
| O | | |  
| X | | |  
| | | X |
```

Turn: O

Emotion detected as neutral (row 0).

Emotion detected as neutral (col 0).

Manual Text Override -> index 1

Player O took position (0, 1).

```
| O | O | |  
| X | | |  
| | | X |
```

Turn: X

Player X took position (1, 1).

```
| O | O | |  
| X | X | |  
| | | X |
```

Turn: O

Emotion detected as neutral (row 0).

Emotion detected as neutral (col 0).

Manual Text Override -> index 2

Player O took position (0, 2).

```
| O | O | O |  
| X | X | |  
| | | X |
```

Player O has won!

The interface worked well and it didn't crash when connecting the game to the webcam. When I played the game, the model only guessed "neutral" for every move. So to complete the game, I had to use the manual terminal text override system instead of my expressions. So, it did not recognize my facial expressions with the same accuracy against the test set. The model scored a high 72.98% accuracy, but it didn't perform well with the live webcam since it didn't detect the "happy" or "surprise" expressions. I think one of the possibilities why it didn't work is because of the lighting in my room. In the training/test expression images, it looks like the lighting was good which detected the expressions better. I also think it's because the model expects closely cropped faces maybe.

Code:

```
def _get_emotion(self, img) -> int:

    #changes the camera image from grayscale --> full RGB color
    img_rgb = cv2.cvtColor(img, cv2.COLOR_GRAY2RGB)

    #shrink/stretch the image to the required size
    img_resized = cv2.resize(img_rgb, image_size)

    #scales pixel numbers to a 0-1 decimal range
    img_normalized = img_resized / 255.0

    final_image = np.expand_dims(img_normalized, axis=0)

    model = models.load_model('results/basic_model_20_epochs_timestamp_1785630672.keras')

    predictions = model.predict(final_image)

    #finds the list index with highest score
    emotion_index = np.argmax(predictions[0])
    return int(emotion_index)
```